Implementación en Java

Creación del proyecto

Una vez definido el diseño pasamos al código. Creamos un proyecto Java con Maven, que es el tipo de proyecto que Apache NetBeans genera por defecto y que Visual Studio Code también reconoce con la extensión *Extension Pack for Java*; así cualquier integrante puede abrirlo en el entorno que prefiera. Las dos clases se ubicaron en el paquete `ec.edu.uta.sistemaacademico`, cuyo nombre sigue la convención de usar el dominio de la institución escrito al revés. La Figura 3 muestra la estructura final del proyecto.

Figura 3

Estructura del proyecto POO-Tema1-Grupo1

```
POO-Tema1-Grupo1/
├── docs/                  ← diagramas de clase y de objetos
├── evidencias/            ← capturas de la ejecución
├── src/main/java/ec/edu/uta/sistemaacademico/
│   ├── Estudiante.java    ← la clase (el molde)
│   └── Principal.java     ← método main: crea y muestra los objetos
├── .gitignore             ← archivos que Git no debe subir
├── pom.xml                ← configuración del proyecto Maven
└── README.md              ← documentación del repositorio
```

Clase Estudiante

La clase `Estudiante` (Figura 4) solo declara los cinco atributos, entre las líneas 22 y 34, y cada uno tiene un comentario que explica qué guarda. Al compilar esta clase todavía no existe ningún estudiante: solo se le indica a Java cómo será cada estudiante cuando se cree.

Figura 4

Fragmento de la clase Estudiante.java: declaración de la clase y de sus atributos

```
15  public class Estudiante {
16
17      // ===== Atributos =====
18      // Son las características que comparten todos los estudiantes. 19
// Cada objeto tendrá su propia copia de estos cinco atributos.
20
21      /** Nombre(s) del estudiante. Ejemplo: "Daniela". */
22      public String nombre;
23
24      /** Apellido(s) del estudiante. Ejemplo: "Villacres". */
25      public String apellido;
26
27      /** Edad en años cumplidos. Ejemplo: 18. */
28      public int edad;
29
30      /** Carrera en la que está matriculado. Ejemplo: "Software". */
31      public String carrera;
32
33      /** Semestre (nivel) que cursa actualmente, del 1 al 10. Ejemplo: 1. */
34      public int semestre; 35  }
```

Nota. Se muestran las líneas 15 a 35; el archivo completo se encuentra en el Apéndice A.

Clase Principal

La clase `Principal` contiene el método `main`, que es el punto de entrada del programa. Su código se organizó en tres bloques, identificados con comentarios para que sea fácil de seguir. La Figura 5 muestra los dos primeros.

Figura 5

Fragmento de la clase Principal.java: instanciación y asignación de valores

```
16      public static void main(String[] args) {
17
18      // ===== 1. Instanciación =====
19      // Con "new" se crean tres objetos distintos a partir de la misma clase.
20      Estudiante estudiante1 = new Estudiante();
21      Estudiante estudiante2 = new Estudiante(); 22          Estudiante estudiante3 = new
Estudiante(); 23
24          // ===== 2. Asignación de valores a los atributos =====
25          // Se usa el operador punto (.) para acceder a cada atributo del objeto.
26          estudiante1.nombre = "Daniela";
27          estudiante1.apellido = "Villacres";
28          estudiante1.edad = 18;
29          estudiante1.carrera = "Software";
30          estudiante1.semestre = 1;
31
32          estudiante2.nombre = "Mateo";
33          estudiante2.apellido = "Naranjo";
34          estudiante2.edad = 20;
35          estudiante2.carrera = "Telecomunicaciones";
36          estudiante2.semestre = 3;
37
38          estudiante3.nombre = "Camila";
39          estudiante3.apellido = "Guerrero";
40          estudiante3.edad = 22;
41          estudiante3.carrera = "Medicina";
42          estudiante3.semestre = 6;
```

Nota. Se muestran las líneas 16 a 42; el archivo completo se encuentra en el Apéndice B.

Cada bloque cumple una función concreta dentro del programa:

1. **Instanciación (líneas 18 a 22).** Se crean tres objetos con `new Estudiante()` y cada uno se guarda en su propia variable: `estudiante1`, `estudiante2` y `estudiante3`.
2. **Asignación de valores (líneas 24 a 42).** Con el operador punto se accede a cada atributo del objeto y se le asigna un valor, por ejemplo `estudiante1.nombre = "Daniela";`. Los tres objetos reciben datos diferentes.
3. **Presentación de la información (desde la línea 44).** Con `System.out.println` se muestra cada atributo, uniendo un texto fijo con el valor del objeto mediante el operador `+`.

Ejecución del programa

Para comprobar el funcionamiento compilamos y ejecutamos el programa desde la terminal. La Figura 6 muestra los comandos utilizados y la salida que se obtuvo.

Figura 6

Ejecución del programa en la terminal



Nota. Salida obtenida al compilar con javac y ejecutar con java en OpenJDK 21.

El comando `javac` compila las dos clases y guarda los archivos `.class` en la carpeta `bin`, y el comando `java` ejecuta la clase `Principal`. La información de los tres estudiantes se muestra completa y en el mismo orden en que se asignaron los valores, lo que confirma que cada objeto conserva sus propios datos. El programa también se puede ejecutar con el botón

Run de Visual Studio Code o con Run Project en NetBeans; los pasos están detallados en el archivo README.md del repositorio.